*Bounds Classes Cheat Sheet

pricing and allocation bounds

Three classes in `aggregate.bounds`, two papers, one question: *given that a risk is priced to P, what else is pinned down?* All three parameterise the extreme consistent distortions as biTVaRs $(1-w_1)\,\mathrm{TVaR}_{p_0} + w_1\,\mathrm{TVaR}_{p_1}$ with $p_0 \le p_* \le p_1$.

Not DecL-creatable — but DecL is at both ends. Every *input* is a DecL object (`build('port ...')`, `build('agg ...')`), and every *output* is a biTVaR distortion, which is DecL-writable: `ab.distortion(P, 'A', 'upper')` returns a `Distortion` you can spell `dist D bitvar <p0> <p1> <w1>` and hand to `port.price`. That round trip is the creation story; the constructors below are the only entry points.

How they differ. `Bounds` studies the distortion family itself (the envelopes of g), on a p-knot grid with a root find: approximate. The two hull classes study the *image* of that family, on exact CDF-breakpoint vertices with monotone convex hulls: exact, and P enters only as a cheap slice.

The geometry. Represent a distortion by its Kusuoka measure μ, so $\rho_\mu(X) = \int \mathrm{TVaR}_p(X)\,\mu(dp)$. Pricing X to P is *one affine constraint* on probability measures, so the extreme consistent measures are two-point mixtures, biTVaRs. Write $T(p) = \mathrm{TVaR}_p(X)$ and plot the score $y_j(p)$ against $T(p)$ as p sweeps $[0,1]$: the vertical slice at $T = P$ through the convex hull of that curve is $[\min, \max]$, and the hull edge crossing $T = P$ names the achieving biTVaR. On the FFT grid both coordinates are affine in $u = 1/(1-p)$ within an atom, so the curve is exactly piecewise linear and the hull of the curve equals the hull of its breakpoint vertices: no p-grid, no interpolation error. Here p_* solves $T(p_*) = P$, exeqa is $\mathsf{E}[X_i \mid X = x]$, and NA is the natural allocation. **The Gini lens** is the case $X = U[0,1]$, where $T(p) = (1+p)/2$ is affine in p, the constraint collapses to the mean Kusuoka level $\mathsf{E}_\mu[p] = 2P - 1$, and the bounds read off the envelope gap of Y's own TVaR curve: an X-free reading of why distortions pricing Y disagree.

The columns show all (m) `methods`, and fields or properties (used interchangeably), under the eleven standard category headings. Categories **2** (update), **3** (moments), **7** (reinsurance), and **10** (approximations) are empty for all three and are omitted: a bounds object is built once from an already-updated risk and is then pure geometry, with nothing to re-discretize, no moments of its own, no cessions, no fitted approximation.

`Bounds` — the distortion family (IME 2022)

(m) `Bounds(obj, premium, *, a=inf, unit='total', n_p=256, n_s=513)`

1. Specification & creation. `obj` is a `Portfolio`, `Aggregate`, or pmf `Series` / `DataFrame`; the premium is *baked in at construction* ($\mathsf{E}[X] < \texttt{premium} \le a$). `name`, `unit`, `premium`, `a` (asset cap), `Fb` ($F(a)$), `n_p`, `n_s` (grid sizes).

4. Statistical functions. `p_star` (the p with $\mathrm{TVaR}_p(\min(X,a)) = \text{premium}$; a cached property, found by root search), `p_knots` (the p axis), `s_grid` (the s axis), `tvar_x_p`, `tvar_hinges` ($\min(1, s/(1-p))$), (m) `distortion(pl, pu)` (the biTVaR with those knots and the matching weight).

5. Validation. *None:* the p-knot grid plus `brentq` is approximate by construction. Refine with `n_p` / `n_s`.

6. Output dataframes. `info`, `tvar_df` (p vs. TVaR), `weight_df` (one row per bracketing (p_0, p_1) pair), `cloud_df` (shape (n_s, K), one column per bracket).

8. Visualization. (m) `plot_envelope` (the three-panel cloud figure from the paper), (m) `plot_weights`.

9. Risk and pricing. `min_envelope` (pointwise minimum of the cloud; min of concaves is concave, so this *is* a `Distortion`), `max_envelope` (pointwise maximum, an interpolating *callable only*: max of concaves need not be concave), `min_envelope_hinges`.

11. Meta. (m) `help`.

`AllocationBounds` — by unit

(m) `AllocationBounds(port, *, a=inf, units=None, s_floor=1e-14)`
(m) `Portfolio.allocation_bounds(*, a=0, p=0, units, s_floor)`

1. Specification & creation. Portfolio only (it reads the `exeqa_*` columns, so the portfolio must be updated). Given the total is priced to P, what is the range of *natural allocation* premium to each unit? `port`, `units`, `a`, `s_floor`. *The premium is a call-time argument, not baked in.*

4. Statistical functions. (m) `p_star(P)` (exact per-atom inversion of $T(p) = P$), (m) `bounds(P)` (the answer: `lower` / `upper` / `width` by unit), (m) `bitvars(P)` (the achieving (p_0, p_1, w_1) per unit and side), (m) `na_grid(p_grid)`.

5. Validation. (m) `check(P)` (audit: reprice with the achieving biTVaRs), `additivity_error` ($\max_p |\sum_i a_i(p) - T(p)|$, inherited `density_df` FFT noise; $\sim 10^{-6}$ relative).

6. Output dataframes. `info`, `curve_df` (vertex table indexed by p: `exeqa_total` $= T(p)$, one `exeqa_<unit>` column each), `premium_range` ($= [\mathsf{E}[X], T_{\max}]$).

8. Visualization. (m) `plot(items, P)` (curves, envelopes, the P-slice). **9. Risk and pricing.** (m) `distortion(P, item, bound)`. **11. Meta.** (m) `help`.

`PricingBounds` — any second risk

(m) `PricingBounds(x_source, y_sources, *, a=inf, s_floor=1e-14, n_grid=1024)`
(m) `Portfolio.pricing_bounds(y_sources, *, a=0, p=0, s_floor, n_grid)`

1. Specification & creation. The same question for *any* second risk: given X is priced to P, what is the range of the price of Y? A source is an `Aggregate`, `Portfolio`, pmf `Series`, or `'uniform'`; `y_sources` takes one, a list, or a naming dict. `a`, `s_floor`, `n_grid`.

4. Statistical functions. (m) `p_star(P)`, (m) `bounds(P)`, (m) `bitvars(P)` *(as for* `AllocationBounds`*)*, (m) `mean_kusuoka_level(P)` $(= 2P - 1)$, `gini_level`.

5. Validation. (m) `check(P)`. *Bounded (a finite) reprices to* $\sim 10^{-12}$. *Unbounded with a heavy-tailed Y the upper bound is genuinely tail-driven and grid-sensitive: cap the risks or raise* `s_floor`.

6. Output dataframes. `info`, `curve_df` (vertex table: `tvar_<X>` plus one `price_<Y>` column per Y), `premium_range`.

8. Visualization. (m) `plot(items, P)`. **9. Risk and pricing.** (m) `distortion(P, item, bound)`. **11. Meta.** (m) `help`.

aggregate v.1.0.0a171
ease | confidence | precision

2026-07-29 11:34